

SERVERLESS EXPENSE TRACKER STATICS WEB SERVER ON AWS

VINEETH KUMAR M

Febuary Batch

3:30PM to 5:00PM

INDEX:

- **INTRODUCTION**
- **AWS SERVICES**
- **DIAGRAM**
- **EXPLANATION**
- **OUTPUT**
- **CONCLUTION**

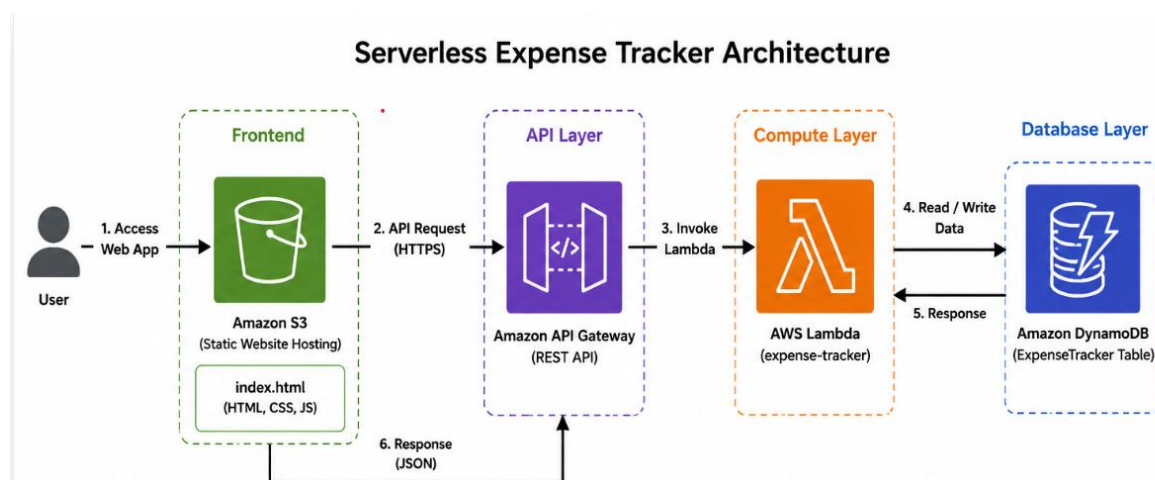
INTRODUCTION:

This project is a Serverless Expense Tracker Web Application developed using Amazon Web Services (AWS) via deployed using. The front-end is hosted via S3 static web. Backend database has connected with Lambda function (python 3.13 version) via API Gateway storing data handled by DynamoDB.

AWS SERVICES:

- 1. DYNAMO DB:** Data is stored in tables using primary keys such as `expense_id`. And integrates easily with AWS services like Lambda and API Gateway.
- 2. LAMBDA FUNCTION:** Create a function for backend server with (python 3.13) to fetch the static web site hosted on S3.
- 3. API GATEWAY:** Create Rest API method to invoke through dynamodb.
- 4. S3 bucket:** Hosted the public frontend web server via (HTML CSS AND PYTHON).

DIAGRAM:



EXPLANATION:

- Hosted the S3 static frontend website for expense tracker.
- Create a LAMBDA function and execute the python code to connect the DB table to store data.
- Created an API resources for expenses.
 - Create post method with Rest API and attached the lambda function and deploy.

INSTRUCTIONS:

1. DYNAMODB

- Create a Dynamodb table for expenses
- Define partition key expensed_id

Tables (1) Info

Find tables

Filter by tag
Any tag key

Filter by tag value
Any tag value

May 16, 2026, 17:39 (UTC+5:30)

⌂

AI

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity m
☐	ExpenseTracker	Active	expense_id (S)	-	0	0	Off	☆	On-demand

Completed - Items returned: 5 - Items scanned: 5 - Efficiency: 100% - RCUs consumed: 2

Table: ExpenseTracker - Items returned (5)

Scan started on May 18, 2026, 22:56:27

⌂

Actions

Create item

< 1 >

⚙

☐	expense_id (String)	amount	category	expense_name
☐	81ad1879-6845-4637-...	200	Food	table
☐	0a2d21b8-1d92-4dee-b...	7300	Others	Computer table
☐	2ab606c1-c78b-4861-b...	100	Food	chair
☐	43b40bfd-6994-42ce-a...	127000	Others	Mobile phone
☐	47d5d91b-ea81-48bf-8...	50000	Shopping	Laptop

2. API GATEWAY:

- Create a Rest API in API Gateway.
- Add Resources and method
 - Post – post the data in Lambda.
 - Enable the lambda proxy integration to send all request.
- Enable CORS and deploy API, API URL has created.

APIs (1/1)

Find APIs

Name	Description	ID	Protocol	API endpoint type	Created
ExpenseTrackerAPI		ubts5fw6gl	REST	Regional	2026-05-16

Resources

Create resource

/expense

OPTIONS

POST

/expense - POST - Method execution

ARN: `arn:aws:execute-api:eu-north-1:803283181476:ubts5fw6gl:POST/expense` Resource ID: `twztp1`

Client → Method request → Integration request → Lambda integration

← Integration response ← Method response

Method request | Integration request | Integration response | Method response | Test

3. Lambda function

- Create a Lambda function deploys the python code.
- Enable the lambda proxy to get the request.

Code | Test | Monitor | Configuration | Aliases | Versions

Code source info

Open in Visual Studio Code | Update

EXPLORE

EXPENSE-TRACKER-FUNCTION

lambda_function.py

```
def lambda_handler(event, context):
    table.put_item(
        Item={
            'expense_id': expense_id,
            'expense_name': expense_name,
            'amount': Decimal(str(amount)),
            'category': category
        }
    )

    response = {
        "message": "Expense Added Successfully",
        "expense_name": expense_name,
        "amount": amount,
        "category": category
    }

    return {
        "statusCode": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "*"
        }
    }
```

DEPLOY

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

TEST EVENTS (SELECTED: TEST)

+ Create new test event

Private

Test

Event JSON

```
{
  "body": "{\"expense_name\":\"Laptop\",\"amount\":\"50000\",\"category\":\"Electronics\"}"
}
```

PROBLEMS | OUTPUT | CODE REFERENCE LOG | TERMINAL

Status: Succeeded

Test Event Name: Test

Response:

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Headers": "*"
  }
}
```

Lambda Deployed | Amazon Q

4. S3 Storage Bucket

- Create a Static front end web page as public.

ex.html

Copy S3 URIDownloadOpen L2Object actions

PropertiesPermissionsVersions

Object overview

Owner

184f94c9e028fc30ba9b396d3c968782eef5966087d985ddb78c18c24ae652

AWS Region

Asia Pacific (Osaka) ap-northeast-3

Last modified

May 18, 2026, 22:50:13 (UTC+05:30)

Size

3.4 KB

Type

html

Key

ex.html

S3 URI

s3://elasticbeanstalk-ap-northeast-3-803283181476/ex.html

Amazon Resource Name (ARN)

arn:aws:s3:::elasticbeanstalk-ap-northeast-3-803283181476/ex.html

Entity tag (Etag)

6a2776206a3823360c843d8

Object URL Copied

https://elasticbeanstalk-ap-northeast-3-803283181476.s3.ap-northeast-3.amazonaws.com/ex.html

Object management overview

The following bucket properties and object management configurations impact the behavior of this object.

Bucket properties

Bucket Versioning

When enabled, multiple variants of an object can be stored in the bucket to easily recover from unintended user actions and application failures.

Disabled

Bucket "elasticbeanstalk-ap-northeast-3-803283181476" doesn't have Bucket Versioning enabled

We recommend that you enable Bucket Versioning to help protect against unintentionally overwriting or deleting objects. [Learn more](#)

Enable Bucket Versioning

Management configurations

Replication status

When a replication rule is applied to an object the replication status indicates the progress of the operation.

-

[View replication rules](#)

Expiration rule

You can use a lifecycle configuration to define expiration rules to schedule the removal of this object after a pre-defined time period.

-

Expiration date

The object will be permanently deleted on this date.

-

5. FRONTEND:

Expense Tracker

Expense Name

Amount

Category

dd - mm - yyyy

Add Expense

Name

Amount

Category

Date

6. OUTPUT

Expense Tracker



Add Expense

Name	Amount	Category	Date
Chair	12000	Houe	2026-05-26
Table	1000	House	2026-05-12
Mobile Phone	143000	Personal use	2026-05-23

CONCLUSION:

The Serverless Expense Tracker application was successfully developed using AWS cloud services such as **Amazon Web Services, Amazon S3, Amazon API Gateway, AWS Lambda, Amazon DynamoDB,**

The frontend was hosted in S3, API Gateway handled REST API requests, Lambda processed the business logic, and DynamoDB stored the expense details.